

TRD

Technical Requirements Document (TRD)

AWS S3-Based Data Processing Pipeline

•

TR-INGEST-001

Related Functional Requirement(s)

FR-INGEST-001

Objective

Implement S3 data ingestion for customer and order CSV files into AWS Glue Spark DataFrames.

Target Cluster Configuration

- Service: AWS Glue ETL Job
- Worker Type: G.1X or G.2X
- Number of Workers: 2-10 (based on data volume)
- Glue Version: 3.0 or higher
- Language: PySpark

Source Data Details

- Customer Data Location: `s3://adif-sdlc/sdlc\_wizard/customerdata/`
- Customer Schema:
 - CustId: string
 - Name: string
 - EmailId: string
 - Region: string
- Order Data Location: `s3://adif-sdlc/sdlc\_wizard/orderdata/`
- Order Schema:
 - OrderId: string
 - ItemName: string
 - PricePerUnit: decimal/double
 - Qty: integer
 - Date: string/date
 - CustId: string

- Format: CSV
- Encoding: UTF-8

Target Data Details

- Target: In-memory Spark DataFrames
- Customer DataFrame: customer_df
- Order DataFrame: order_df

Job Flow / Pipeline Stages

1. Initialize Glue context and Spark session
2. Read customer CSV files from S3 using Glue DynamicFrame or Spark DataFrame
3. Apply customer schema during read operation
4. Read order CSV files from S3 using Glue DynamicFrame or Spark DataFrame
5. Apply order schema during read operation
6. Validate data load success (record count > 0)

Data Transformations / Business Logic

- No transformations at this stage
- Direct CSV-to-DataFrame mapping with schema enforcement

Error Handling and Logging

- Log start and completion of each read operation to CloudWatch
- Capture and log S3 access errors (permissions, missing files)
- Validate non-empty DataFrames; fail job if source data is missing
- Log record counts for customer and order datasets

Assumptions

- CSV files use comma delimiter
- Header row is present in all CSV files
- All required columns exist in source files
- File encoding is UTF-8 compatible
-

TR-CLEAN-001

Related Functional Requirement(s)

FR-CLEAN-001

Objective

Remove records containing "Null" string literals or actual NULL values from customer and order datasets.

Target Cluster Configuration

- Service: AWS Glue ETL Job (same job as TR-INGEST-001)
- Execution: Sequential stage after ingestion

Source Data Details

- Input: customer_df and order_df from TR-INGEST-001

Target Data Details

- Output: customer_clean_df and order_clean_df
- Schema: Unchanged from source

Job Flow / Pipeline Stages

1. Filter customer_df to exclude rows where any column contains string "Null" or is NULL
2. Store result in customer_clean_df
3. Filter order_df to exclude rows where any column contains string "Null" or is NULL
4. Store result in order_clean_df
5. Log record counts before and after cleaning

Data Transformations / Business Logic

- Apply filter condition: `col != "Null" AND col IS NOT NULL` for all columns
- Use Spark DataFrame filter or SQL WHERE clause
- Case-sensitive match for string "Null"

Error Handling and Logging

- Log record counts before and after null removal for both datasets
- Log percentage of records removed
- Warn if more than 50% of records are removed (potential data quality issue)

Assumptions

- "Null" is case-sensitive string literal
- NULL refers to Spark null values
- No partial row retention; entire row is removed if any column fails condition
-

TR-CLEAN-002

Related Functional Requirement(s)

FR-CLEAN-002

Objective

Remove duplicate records from customer and order datasets based on all columns.

Target Cluster Configuration

- Service: AWS Glue ETL Job (same job as TR-CLEAN-001)
- Execution: Sequential stage after null removal

Source Data Details

- Input: customer_clean_df and order_clean_df from TR-CLEAN-001

Target Data Details

- Output: customer_dedup_df and order_dedup_df
- Schema: Unchanged from source

Job Flow / Pipeline Stages

1. Apply Spark `dropDuplicates()` on customer_clean_df across all columns
2. Store result in customer_dedup_df
3. Apply Spark `dropDuplicates()` on order_clean_df across all columns
4. Store result in order_dedup_df
5. Log record counts before and after deduplication

Data Transformations / Business Logic

- Use Spark DataFrame `dropDuplicates()` method without column specification (all columns)
- Retain first occurrence of duplicate set (Spark default behavior)

Error Handling and Logging

- Log record counts before and after deduplication for both datasets
- Log number of duplicates removed
- Warn if duplicate rate exceeds 10%

Assumptions

- No specific ordering preference for which duplicate to retain
- Duplicates are defined as exact match across all columns
- Deduplication is performed independently for each dataset
-

TR-CATALOG-001

Related Functional Requirement(s)

FR-CATALOG-001

Objective

Register cleaned customer and order datasets as Glue Data Catalog tables.

Target Cluster Configuration

- Service: AWS Glue ETL Job (same job as TR-CLEAN-002)
- Execution: Sequential stage after deduplication

Source Data Details

- Input: customer_dedup_df and order_dedup_df from TR-CLEAN-002

Target Data Details

- Database: `gen\_ai\_poc\_databrickscoe`
- Customer Table: `sdlc\_wizard\_customer`
- CustId: string
- Name: string
- EmailId: string
- Region: string
- Order Table: `sdlc\_wizard\_order`
- OrderId: string
- ItemName: string
- PricePerUnit: decimal/double
- Qty: integer
- Date: string/date
- CustId: string
- Storage Format: Parquet (recommended) or CSV
- Storage Location:
- Customer: `s3://adif-sdlc/curated/sdlc\_wizard/customer/`
- Order: `s3://adif-sdlc/curated/sdlc\_wizard/order/`

Job Flow / Pipeline Stages

1. Write customer_dedup_df to S3 in Parquet format
2. Create or update Glue Catalog table `sdlc\_wizard\_customer` pointing to customer data location
3. Write order_dedup_df to S3 in Parquet format
4. Create or update Glue Catalog table `sdlc\_wizard\_order` pointing to order data location
5. Verify table registration in Glue Catalog

Data Transformations / Business Logic

- Convert DataFrames to Glue DynamicFrames if using Glue API
- Use `getCatalogSink()` or `write.format("parquet").save()` with catalog update

Error Handling and Logging

- Log S3 write operations with record counts
- Log Glue Catalog table creation/update status
- Capture and log permission errors for catalog operations
- Verify table metadata is queryable post-registration

Assumptions

- Database `gen\_ai\_poc\_databrickscoe` exists prior to job execution
- Tables are created if not exist, or overwritten if exist
- IAM role has `glue:CreateTable` and `glue:UpdateTable` permissions
-

TR-SCD2-001

Related Functional Requirement(s)

FR-SCD2-001

Objective

Implement SCD Type 2 logic for ordersummary table using Apache Hudi with customer-order join.

Target Cluster Configuration

- Service: AWS Glue ETL Job
- Worker Type: G.1X or G.2X
- Number of Workers: 5-20 (based on data volume)
- Glue Version: 3.0 or higher with Hudi support
- Language: PySpark
- Hudi Version: 0.10.0 or higher

Source Data Details

- Input Tables:
- `gen\_ai\_poc\_databrickscoe.sdmc\_wizard\_customer`
- `gen\_ai\_poc\_databrickscoe.sdmc\_wizard\_order`
- Join Key: CustId

Target Data Details

- Database: `gen\_ai\_poc\_databrickscoe`
- Table: `ordersummary`
- Schema:
- CustId: string
- Name: string
- EmailId: string
- Region: string
- OrderId: string
- ItemName: string
- PricePerUnit: decimal/double
- Qty: integer
- Date: string/date
- IsActive: boolean
- StartDate: timestamp
- EndDate: timestamp (nullable)
- OpTs: timestamp
- Storage Location: `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`
- Storage Format: Apache Hudi (Copy-on-Write or Merge-on-Read)
- Hudi Configuration:
- Table Type: COPY_ON_WRITE
- Record Key: OrderId
- Precombine Field: OpTs
- Partition Path: Date (optional)

Job Flow / Pipeline Stages

1. Read current `ordersummary` Hudi table from S3 (if exists)
2. Read `sdlc\_wizard\_customer` and `sdlc\_wizard\_order` from Glue Catalog
3. Perform inner join on CustId
4. Add SCD2 columns: IsActive=true, StartDate=current_timestamp, EndDate=null, OpTs=current_timestamp
5. Compare joined data with existing active records in `ordersummary`
6. For new records: insert with IsActive=true
7. For changed records:
 - Update existing active record: set IsActive=false, EndDate=current_timestamp
 - Insert new record with IsActive=true, StartDate=current_timestamp
8. Write result to S3 using Hudi upsert operation

9. Update Glue Catalog table metadata

Data Transformations / Business Logic

- Join Logic: INNER JOIN customer ON order.CustId = customer.CustId
- Change Detection: Compare all business columns (Name, EmailId, Region, ItemName, PricePerUnit, Qty) between new and existing active records
- SCD2 Logic:
 - New record: No matching OrderId in current active records
 - Changed record: Matching OrderId exists but business column values differ
 - Unchanged record: Matching OrderId with identical business values (no action)
- Timestamp Generation: Use `current\_timestamp()` for StartDate, EndDate, OpTs

Error Handling and Logging

- Log join record count
- Log counts for new, changed, and unchanged records
- Capture Hudi write errors and log to CloudWatch
- Validate Hudi commit success
- Log S3 write location and record count
- Handle initial load scenario (empty ordersummary table)

Assumptions

- OrderId uniquely identifies each order record
- Change detection compares all business columns from customer and order
- OpTs is used as precombine field for Hudi conflict resolution
- Initial load treats all records as new (IsActive=true)
- Hudi table is registered in Glue Catalog with appropriate SerDe
-

TR-SCD2-002

Related Functional Requirement(s)

FR-SCD2-002

Objective

Implement incremental SCD2 updates triggered by customer data changes.

Target Cluster Configuration

- Service: AWS Glue ETL Job (same configuration as TR-SCD2-001)
- Trigger: Scheduled or event-driven (S3 event notification)

Source Data Details

- Input:
- New/changed customer records from `s3://adif-sdlc/sdlc\_wizard/customerdata/`
- Existing `sdlc\_wizard\_order` table
- Existing `ordersummary` Hudi table

Target Data Details

- Output: Updated `ordersummary` Hudi table (same as TR-SCD2-001)

Job Flow / Pipeline Stages

1. Identify new or changed customer records since last job run (using watermark or timestamp)
2. Extract affected CustId values
3. Filter order records matching affected CustId values
4. Execute SCD2 logic (TR-SCD2-001) for filtered customer-order combinations
5. Perform Hudi upsert operation for affected records only
6. Update job bookmark or watermark for next incremental run

Data Transformations / Business Logic

- Change Detection: Compare customer records against previous snapshot or use timestamp column
- Incremental Filter: `WHERE CustId IN (changed\_customer\_ids)`
- SCD2 Logic: Same as TR-SCD2-001, applied to subset of data

Error Handling and Logging

- Log number of changed customers detected
- Log number of affected orders
- Log incremental update record count
- Capture and log watermark/bookmark values
- Handle scenario where no changes are detected (skip processing)

Assumptions

- Customer data includes timestamp or version column for change detection
- Job bookmark or external watermark mechanism tracks last processed state
- Order data is relatively stable; customer changes are primary trigger
- Incremental processing reduces compute cost and execution time
-

TR-AGG-001

Related Functional Requirement(s)

FR-AGG-001

Objective

Generate and maintain customer aggregate spend table from active ordersummary records.

Target Cluster Configuration

- Service: AWS Glue ETL Job
- Worker Type: G.1X or G.2X
- Number of Workers: 2-5
- Glue Version: 3.0 or higher
- Language: PySpark

Source Data Details

- Input: `gen\_ai\_poc\_databrickscoe.ordersummary` (Hudi table)
- Filter: IsActive = true

Target Data Details

- Database: `gen\_ai\_poc\_databrickscoe`
- Table: `customeraggregatespend`
- Schema:
- Name: string
- TotalAmount: decimal/double
- Date: string/date
- Storage Location: `s3://adif-sdlc/analytics/customeraggregatespend/`
- Storage Format: Parquet
- Write Mode: Overwrite (full refresh)

Job Flow / Pipeline Stages

1. Read `ordersummary` Hudi table from S3
2. Filter records where IsActive = true
3. Calculate TotalAmount: `sum(PricePerUnit Qty)` grouped by Name
4. Add Date column (current date or max transaction date)
5. Write aggregated results to S3 in Parquet format
6. Update Glue Catalog table `customeraggregatespend`

Data Transformations / Business Logic

- Aggregation: `GROUP BY Name`

- Calculation: `SUM(PricePerUnit Qty) AS TotalAmount`
- Date Logic: Use `current\_date()` or `MAX(Date)` from source records

Error Handling and Logging

- Log number of active records processed
- Log number of unique customers in output
- Log total aggregate amount across all customers
- Capture and log S3 write errors
- Validate non-zero output record count

Assumptions

- Only active records (IsActive=true) are included in aggregation
- TotalAmount represents lifetime spend per customer
- Date represents aggregation run date or latest transaction date
- Full refresh approach is acceptable (no incremental aggregation)
-

TR-INFRA-001

Related Functional Requirement(s)

FR-INFRA-001

Objective

Configure AWS Glue infrastructure with Spark and Hudi for ETL processing.

Target Cluster Configuration

- Service: AWS Glue
- Glue Version: 3.0 or higher
- Job Type: Spark (PySpark)
- Worker Type: G.1X (4 vCPU, 16 GB memory) or G.2X (8 vCPU, 32 GB memory)
- Number of Workers: 2-20 (auto-scaling enabled)
- Max Concurrent Runs: 1 (for SCD2 jobs to avoid conflicts)
- Timeout: 2880 minutes (48 hours)
- Max Retries: 0 (manual retry preferred for data consistency)

Source Data Details

- S3 Buckets:
- `s3://adif-sdlc/sdlc\_wizard/customerdata/`
- `s3://adif-sdlc/sdlc\_wizard/orderdata/`

- `s3://adif-sdlc/curated/sdlc\_wizard/`
- `s3://adif-sdlc/analytics/`

Target Data Details

- Glue Data Catalog Database: `gen\_ai\_poc\_databrickscoe`
- CloudWatch Log Group: `/aws-glue/jobs/`

Job Flow / Pipeline Stages

1. Configure Glue job with required IAM role
2. Add Hudi library as extra JAR or use Glue marketplace connector
3. Set Spark configuration parameters for Hudi
4. Enable CloudWatch logging
5. Configure job bookmarks for incremental processing (where applicable)
6. Set up Glue Catalog as Hive metastore

Data Transformations / Business Logic

- Hudi Configuration Parameters:
- `hoodie.table.name`: ordersummary
- `hoodie.datasource.write.recordkey.field`: OrderId
- `hoodie.datasource.write.precombine.field`: OpTs
- `hoodie.datasource.write.operation`: upsert
- `hoodie.datasource.write.table.type`: COPY_ON_WRITE
- `hoodie.datasource.hive\_sync.enable`: true
- `hoodie.datasource.hive\_sync.database`: gen_ai_poc_databrickscoe
- `hoodie.datasource.hive\_sync.table`: ordersummary
- `hoodie.datasource.hive\_sync.mode`: hms

Error Handling and Logging

- Enable CloudWatch Logs with INFO or DEBUG level
- Log job start, end, and duration
- Capture Spark executor logs
- Log Hudi commit metadata
- Set up CloudWatch alarms for job failures
- Enable Glue job metrics (DPU usage, data processed)

Assumptions

- IAM role includes policies for:
- S3 read/write access to specified buckets

- Glue Catalog read/write access
- CloudWatch Logs write access
- Hudi library version is compatible with Glue version
- No Lake Formation governance is applied initially
- Glue Data Catalog serves as Hive metastore for Hudi
-

TR-QUERY-001

Related Functional Requirement(s)

FR-QUERY-001

Objective

Enable Amazon Athena querying for all Glue Catalog tables including Hudi tables.

Target Cluster Configuration

- Service: Amazon Athena
- Workgroup: Primary or custom workgroup
- Query Result Location: `s3://adif-sdlc/athena-results/`

Source Data Details

- Glue Catalog Database: `gen\_ai\_poc\_databrickscoe`
- Tables:
 - `sdlc\_wizard\_customer`
 - `sdlc\_wizard\_order`
 - `ordersummary` (Hudi)
 - `customeraggregatespend`

Target Data Details

- Query Interface: Athena Console, JDBC/ODBC, or AWS SDK
- Output Format: CSV, JSON, or Parquet (query result)

Job Flow / Pipeline Stages

1. Verify Glue Catalog table metadata is complete
2. Configure Athena workgroup with S3 output location
3. Test SELECT queries against each table
4. Validate Hudi table reads with appropriate SerDe
5. Verify query results match expected data

Data Transformations / Business Logic

- Hudi Table Query: Use Athena Hudi connector or native Hudi support
- Sample Queries:
 - `SELECT FROM sdlc\_wizard\_customer LIMIT 10;`
 - `SELECT FROM ordersummary WHERE IsActive = true;`
 - `SELECT Name, TotalAmount FROM customeraggregatespend ORDER BY TotalAmount DESC;`

Error Handling and Logging

- Log query execution time and data scanned
- Capture and log Athena query errors
- Validate Hudi SerDe configuration if queries fail
- Monitor query costs via CloudWatch metrics

Assumptions

- Athena supports Hudi table format with Glue Catalog integration
- Hudi tables use read-optimized view for Athena queries (COPY_ON_WRITE)
- Query performance is acceptable without partitioning initially
- Users have appropriate IAM permissions for Athena and S3 access
-

TR-IAM-001

Related Functional Requirement(s)

FR-INFRA-001 (implicit)

Objective

Define IAM roles and policies for Glue jobs and Athena access.

Target Cluster Configuration

- Service: AWS IAM
- Role Name: `GlueETLRole` (example)

Source Data Details

N/A

Target Data Details

- IAM Policy Permissions:
 - S3: `s3:GetObject`, `s3:PutObject`, `s3:DeleteObject`, `s3:ListBucket` on `s3://adif-sdlc/`

- Glue: `glue:GetDatabase`, `glue:GetTable`, `glue:CreateTable`, `glue:UpdateTable`, `glue:GetPartitions`
- CloudWatch: `logs:CreateLogGroup`, `logs:CreateLogStream`, `logs:PutLogEvents`
- Athena: `athena:StartQueryExecution`, `athena:GetQueryExecution`, `athena:GetQueryResults`

Job Flow / Pipeline Stages

1. Create IAM role for Glue with trust policy for `glue.amazonaws.com`
2. Attach managed policy `AWSGlueServiceRole`
3. Create custom inline policy for S3 bucket access
4. Attach policy to Glue job configuration
5. Create separate IAM role or user for Athena access with read-only permissions

Data Transformations / Business Logic

N/A

Error Handling and Logging

- Validate IAM role attachment to Glue jobs
- Test permissions before job execution
- Log permission errors during job runs

Assumptions

- IAM roles follow least privilege principle
- S3 bucket policies do not conflict with IAM policies
- No cross-account access is required
- MFA is not required for service roles
-

GLOBAL TECHNICAL ASSUMPTIONS

1. AWS Environment:

- All services are in the same AWS region
- VPC and networking are configured for Glue access to S3
- S3 buckets have versioning disabled (or versioning is handled externally)

2. Data Quality:

- CSV files are well-formed with consistent delimiters
- Schema drift is not expected; schemas are stable
- Data volumes are within Glue and Athena processing limits (< 10 TB per job)

3. Processing Model:

- Batch processing is acceptable; no real-time requirements
- Job execution frequency is daily or on-demand
- Data latency of up to 24 hours is acceptable

4. Hudi Configuration:

- COPY_ON_WRITE table type is used for ordersummary
- Hudi compaction and cleaning are managed automatically
- No custom Hudi indexing is required initially

5. Error Handling:

- Jobs fail fast on critical errors (missing source data, permission errors)
- Partial failures in SCD2 logic are not retried automatically
- Manual intervention is required for data quality issues

6. Cost and Performance:

- Cost optimization is deferred to post-implementation phase
- No specific SLA for job execution time
- Auto-scaling is enabled but not fine-tuned initially

7. Security:

- Data is not encrypted at rest initially (S3 default encryption acceptable)
- Data in transit uses HTTPS
- No PII masking or tokenization is required

8. Monitoring:

- CloudWatch Logs retention is set to 7 days
- No custom dashboards or alarms are configured initially
- Glue job metrics are monitored via AWS Console

9. Data Retention:

- No archival or deletion policies are defined
- Historical SCD2 records are retained indefinitely
- S3 lifecycle policies are not configured

10. Schema Evolution:

- Schema changes require manual job updates
- No automatic schema evolution is supported
- Backward compatibility is not guaranteed for schema changes